Providing Atomic Sector Updates in Software for Persistent Memory

Vishal Verma

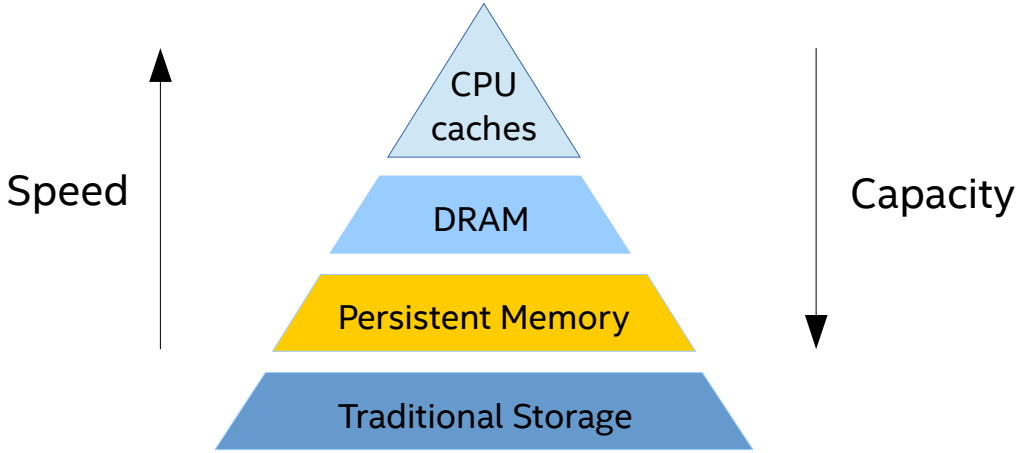
vishal.l.verma@intel.com

Linux Plumbers Conference 2015 – Persistent Memory Microconference

intel Software

Intel OpenSource TECHNOLOGY CENTER

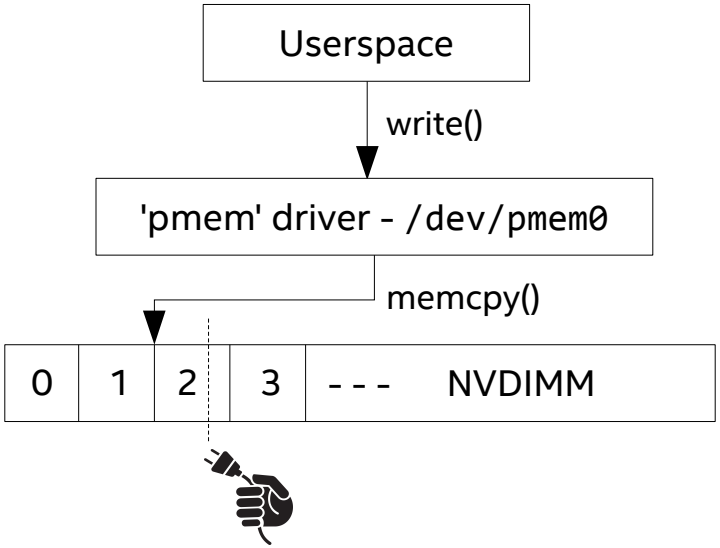
NVDIMMs and Persistent Memory

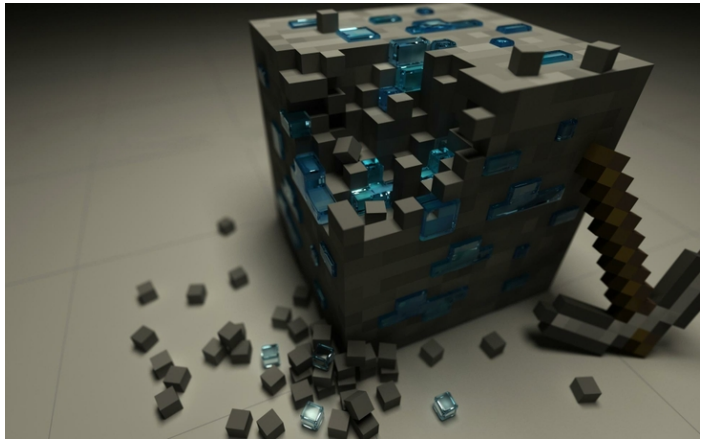


- NVDIMMs are byte-addressable
- Drivers to present these as a block device - “pmem”

Problem Statement

- Byte addressability is great
  - But not for writing a sector atomically





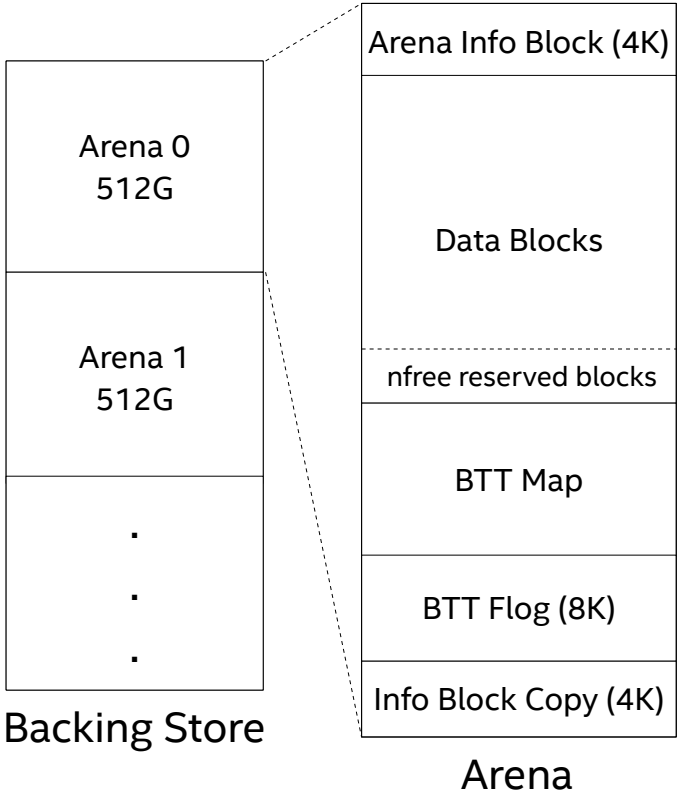


The Block Translation Table

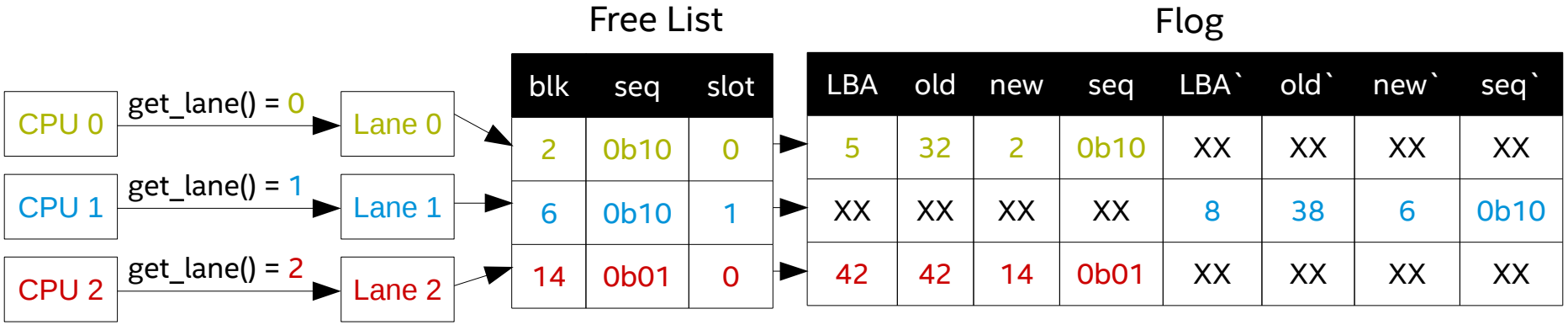
- **nfree:** The number of free blocks in reserve.
- **Flog:** Portmanteau of free list + log
  - Has nfree entries.
  - Each entry has two 'slots' that 'flip-flop'
  - Each slot has:

Block being written	Old mapping	New mapping	Sequence num

- **Info block:** Info about arena - offsets, lbasizes etc.
- **External LBA:** LBA as visible to upper layers
- **ABA:** Arena Block Address - Block offset within an arena
- **Premap/Postmap ABA:** The block offset into the data area as seen prior to/post indirection from the map



The Block Translation Table



- The idea of “lanes” is purely logical
- num_lanes = min(num_cpus, nfree)
- lane = cpu % num_lanes
- If num_cpus > num_lanes, we need locking on lanes
  - But if not, we can simply preempt_disable() and need not take a lock

Map

5	2
8	6
42	14

Performance setup

- This stuff is (will be) in 4.2!
- This stuff is done on Reserved DRAM! (No special hardware)
- Minor changes needed in pmem:
  - Disable direct_access
  - Force NT writes
- These + scripts at:
  - https://git.kernel.org/cgit/linux/kernel/git/vishal/nvdimm.git/log/?h=perf_hacks
- Testing using fio, perf

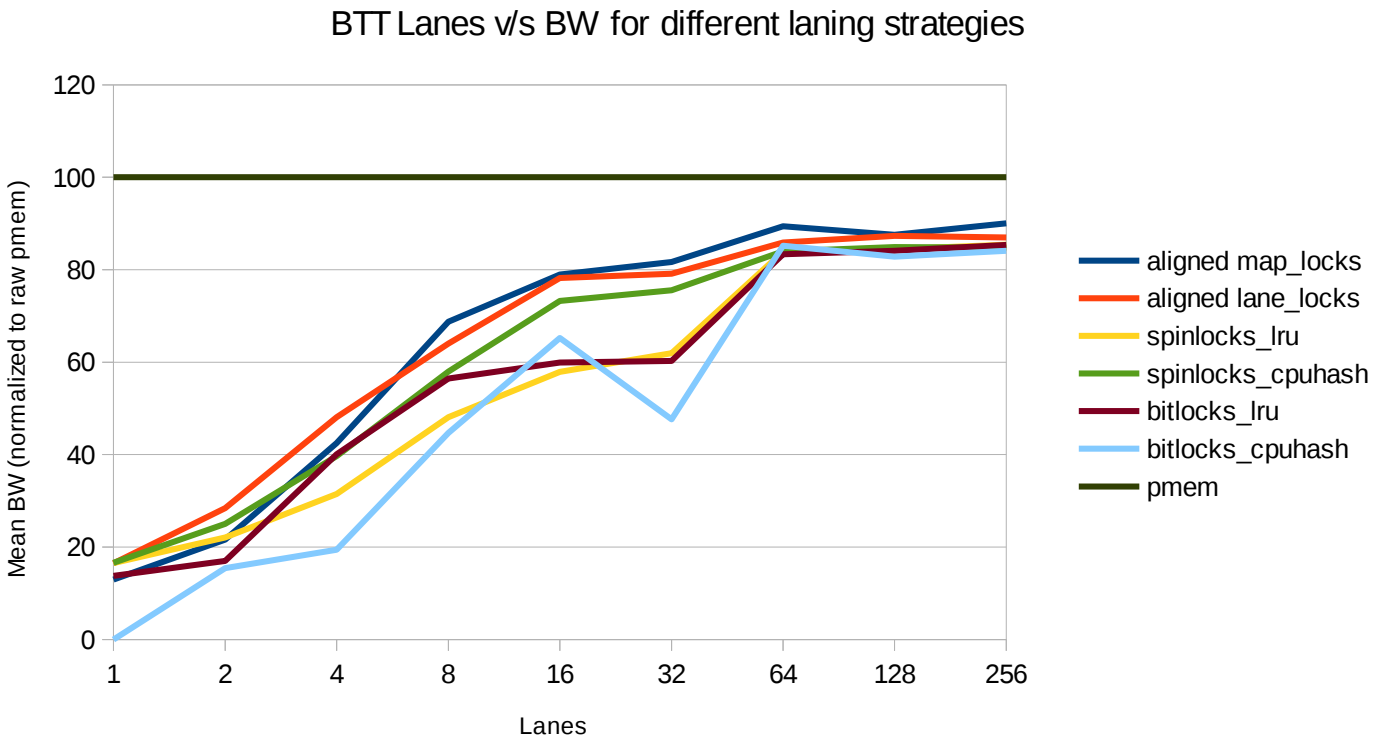
That's Great...but is it Fast?

	512B Block size	4K Block size
Write Amplification	~4.6% [536B]	~0.5% [4120B]
Capacity Overhead	~0.8%	~0.1%

- Comparison to pmem (4K sector size, 32 cores, DRAM backed)
  - Best case = 10% overhead
  - Worst case = 25% overhead
  - Writes take a greater hit – more metadata involved

Performance of lane locking strategies

- Actual = 32 CPUs
- Simulate 64 CPUs
- < 64 → contention ↑
- >= 64 → no locking
- 32 → use locks but no contention



Lets look at perf profiles

- `$ perf record -g dd if=/dev/zero of=/dev/pmem0s bs=4096 count=100000 oflag=direct`

```
+   16.76%    16.76%  dd  [k] __copy_user_nocache
+   12.28%    12.28%  dd  [k] btt_write_pg
+   11.50%    11.50%  dd  [k] __clear_user
+    8.58%     8.58%  dd  [k] _raw_spin_unlock_irqrestore
+    5.26%     5.26%  dd  [k] do_blockdev_direct_IO
+    3.31%     3.31%  dd  [k] kmem_cache_free
+    2.73%     2.73%  dd  [k] _raw_spin_lock
+    2.53%     2.53%  dd  [k] get_user_pages_fast
...
```

Lets look at perf profiles

- $ perf record -g dd if=/dev/zero of=/dev/pmem0s bs=4096 count=100000 oflag=direct

```
+   16.76%    16.76%  dd  [k] __copy_user_nocache
+   12.28%    12.28%  dd  [k] btt_write_pg
+   11.50%    11.50%  dd  [k] __clear_user
+    8.58%     8.58%  dd  [k] _raw_spin_unlock_irqrestore
+    5.26%     5.26%  dd  [k] do_blockdev_direct_IO
+    3.31%     3.31%  dd  [k] kmem_cache_free
+    2.73%     2.73%  dd  [k] _raw_spin_lock
+    2.53%     2.53%  dd  [k] get_user_pages_fast
...
```

Lets look at perf profiles

- $ perf record -g dd if=/dev/zero of=/dev/pmem1 bs=4096 count=100000 oflag=direct

```
+   23.08%    23.08%  dd  [k] __copy_user_nocache
+   14.32%    14.32%  dd  [k] __clear_user
+    9.81%     9.81%  dd  [k] _raw_spin_unlock_irqrestore
+    8.49%     8.49%  dd  [k] do_blockdev_direct_IO
+    5.04%     5.04%  dd  [k] entry_SYSCALL_64_after_swapgs
+    4.24%     4.24%  dd  [k] get_user_pages_fast
+    1.86%     1.86%  dd  [k] __srcu_read_lock
...
```

Lets look at perf profiles

```
static void dio_bio_end_io(struct bio *bio, int error)
{
        struct dio *dio = bio->bi_private;
        unsigned long flags;

        spin_lock_irqsave(&dio->bio_lock, flags);
        bio->bi_private = dio->bio_list;
        dio->bio_list = bio;
        if (--dio->refcount == 1 && dio->waiter)
                wake_up_process(dio->waiter);
        spin_unlock_irqrestore(&dio->bio_lock, flags);
}
```

Ideas?

